
Task W06: Stacks and Queues

The Fast and the Queueous

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Overview

A single-lane parking garage can hold a limited number of cars. Because the garage is only one lane wide, cars park directly behind one another: the first car to arrive ends up at the very back, and each car after it pulls in ahead of the last one. There is only one exit, so a car can only leave by pulling forward and out through it. If a car wants to leave and it is not the one nearest the exit, every car in front of it has to pull out of the garage temporarily, let it out, and then pull back in.

If the garage is completely full when a new car arrives, that car cannot park anywhere. It waits outside, and it is only fair that whichever car has been waiting the longest gets the next open spot once one appears.

2 Choosing Your Structures

This lab requires both of our reference-based Stack Queue classes (included in the lab folder). Reread the overview above and think through each of the following before you start coding: how the garage lane itself should be represented, how the cars waiting outside for a spot should be represented, and how you will correctly handle the cars that have to pull out of the way when the car leaving is not the one nearest the exit. That third case does not necessarily call for a new kind of structure, but it does call for deciding whether the tool you already chose for the lane can also do this job, or whether something else is a better fit.

Get the reasoning right and the code follows naturally. Guess without thinking it through, and you will likely find the cars coming back in the wrong order, or the wrong car getting the next open spot.

3 Required Operations

The garage itself does not come with an arrive or a leave operation, and neither of your data structures does either. These are two new methods that belong in your main program, not inside the Stack or Queue class itself, built out of the push, pop, peek, isEmpty, enqueue, and dequeue operations the data structures already give you.

Before you write them, add a `size()` method to both your Stack and your Queue, backed by a count field that goes up on push/enqueue and down on pop/dequeue. This is the only change you should make to either data structure for this lab. Keep in mind that `size()` only reports how many items are currently stored; it does not enforce a limit or refuse an operation. Deciding what counts as “full” and what to do about it belongs entirely to your main program, in `arrive`, not to the Stack or Queue class itself.

- **arrive(car)**. A car, identified by name, arrives. If the garage has an open spot, it parks directly. Otherwise, it joins the back of the waiting line.
- **leave(car)**. A specific car, identified by name, wants to leave.
 - If the car is not currently parked in the garage (it may be waiting outside, or there may be no record of it at all), do not crash or halt the program; you will continue processing the input until the end of the file. Print `LEAVE <car> (not found)` in place of the usual event line, print the lane and waiting line exactly as they were, unchanged, and move on to the next event.
 - If the car is parked, but not nearest the exit, temporarily remove the cars in front of it, let the requested car exit, and then return the temporarily removed cars to the garage in their original relative order.
 - After a car leaves and a spot opens up, if anyone is waiting outside, the car that has waited longest should be admitted immediately, using the same logic as a normal arrival.

After every event, whether it succeeded or not, print exactly three lines: the event itself (for example, `ARRIVE CarA` or `LEAVE CarA`), then the lane from back to exit on a line starting with `LANE:`, then the waiting line from front to back on a line starting with `WAIT:`. Use a single space between car names. If either is empty, print nothing after the colon. See the worked example below for the exact format. The `display()` methods already in your Stack and Queue classes almost certainly print in a different format than this, so you will likely need to make minor edits to their print statements to get output that matches. You should not need to make any larger changes to the data structures themselves.

At the start of the program, prompt the person running it for the name of the event file to read and for the garage’s capacity, and read both before processing any events. This program should work for any capacity greater than 0.

4 Input File Format

Your program should read a sequence of events from a plain text file. The provided project has a `files` folder containing example event files for this lab; when your program prompts for the file name, the person running it will type just the file’s name, for example `sample_events_07.txt`, not its path. Your program needs to build the full path to the `files` folder itself before opening it. Each line of the file contains one event, either

```
ARRIVE <car>
LEAVE <car>
```

Blank lines should be ignored. Process the file one line at a time, in order, exactly as if the events were happening one after another in real time.

5 A Worked Example

Below is a small example, entering **3** at the capacity prompt, so you can check your program's behavior before running it on the real data file. The events are in `sample_events_07.txt`.

```
1 ARRIVE CarA
2 ARRIVE CarB
3 ARRIVE CarC
4 ARRIVE CarD
5 ARRIVE CarE
6 LEAVE CarA
7 LEAVE CarD
```

Your output should look like this. This is the exact format expected: one block of three lines per event.

```
Enter event file: sample_events_07.txt
Enter garage capacity: 3
ARRIVE CarA
LANE: CarA
WAIT:
ARRIVE CarB
LANE: CarA CarB
WAIT:
ARRIVE CarC
LANE: CarA CarB CarC
WAIT:
ARRIVE CarD
LANE: CarA CarB CarC
WAIT: CarD
ARRIVE CarE
LANE: CarA CarB CarC
WAIT: CarD CarE
LEAVE CarA
LANE: CarB CarC CarD
WAIT: CarE
LEAVE CarD
LANE: CarB CarC CarE
WAIT:
```

Notice that after **CarA** leaves, **CarC** and **CarB** return to the lane in the same relative order they were in before (**CarB** still behind **CarC**), not swapped, and that **CarD** is pulled in from the waiting line automatically once the spot opens up.

Two more files are provided for you to test with before running the real data file: `test_events_50.txt`, with at least 50 events, and `test_events_100.txt`, with at least 100 events. Use them to check that your program holds up once there is a lot going on, especially the displacement logic and the waiting line. I may use any capacity greater than 0 and different data files to test your program.

6 Project Setup and Submission

You have been given a folder named `labw06` with the project already set up: for Java, this is an Eclipse project with the package structure already in place; for C/C++, this is the source layout together with the Makefile. Do your work inside this folder as given. Do not rename it or rearrange

its structure, the layout is part of what gets submitted.

The driver (GarageDriver.java for Java, the equivalent main for C/C++) is a shell for you to complete. You are expected to write the prompts for the event file and the garage capacity, read the event file the person enters from the project's **files** folder, and implement arrive and leave using the provided reference-based Stack and Queue classes, producing the LANE/WAIT output after each event.

If you are working in Java, submit the entire labw06 Eclipse project, not just the individual .java files.

If you are working in C/C++, submit the entire labw06 folder, including the Makefile and the rest of the project layout, not just the Stack and Queue header files on their own.

Every source file you submit needs the required header comment at the top, per course policy.

At the top of your main method, add a comment block explaining which data structures you used and why (a few specific lines using the correct terminology will likely suffice; vague answers will not).

Looking ahead

A future version of this lab could allow a car waiting outside to give up and leave the line before ever getting a spot. If it is the only car waiting, that is easy. But if several cars are waiting and the one that gives up is not at the front, you would need to remove a value from somewhere in the middle of whichever structure you used to hold cars waiting outside, while preserving everyone else's order and position, which is a genuinely tricky operation on a reference-based structure. A future version could also give the waiting line itself a limited size, so that it could fill up too, the way the garage already can. Neither of these is required for this lab, but keep them in mind. You may see one of them again.